MiniMalloc: A Lightweight Memory Allocator for Hardware-Accelerated Machine Learning

Michael D. Moffitt

Google

Presented at the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS 2024)

Rapid Evolution in Machine Learning **Model Architectures**

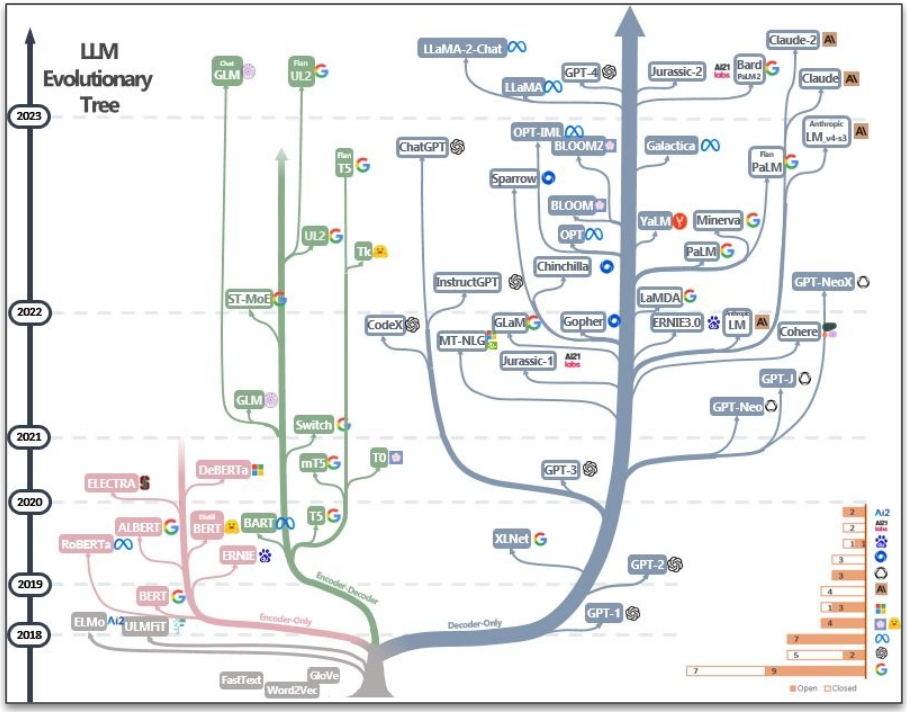
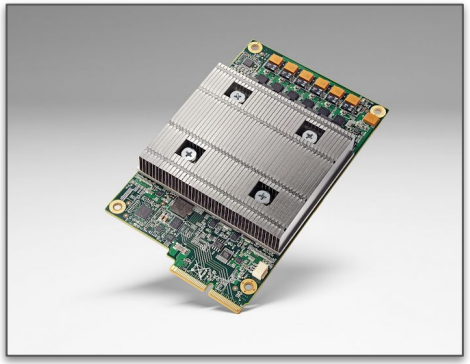
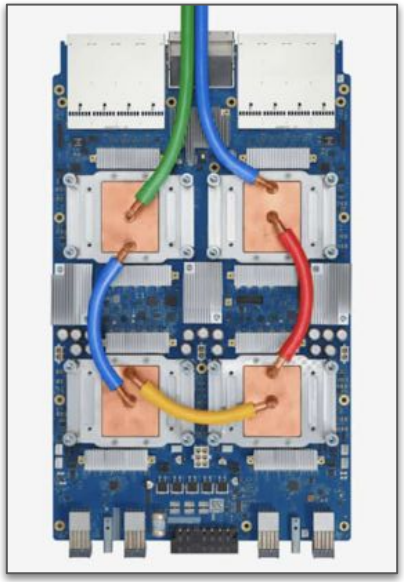


Image Credit: "*Harnessing the Power of LLMs in Practice: A Survey on ChatGPT and Beyond*" by Yang et al., ACM Transactions on Knowledge Discovery from Data, April 2024

Progress in ML Enabled by an Evolution in **Machine Architectures**



Tensor Processing Unit board



A single Cloud TPU v3 device

A Cloud TPU v3 Pod

Image Credit:
https://cloud.google.com/blog/products/ai-machine-learning/google-supercharges-machine-learning-tasks-with-custom-chip
https://cloud.google.com/blog/products/ai-machine-learning/better-scalability-with-cloud-tpu-pods-and-tensorflow-2-1

Compilers are Critical to Maximizing Accelerator Performance

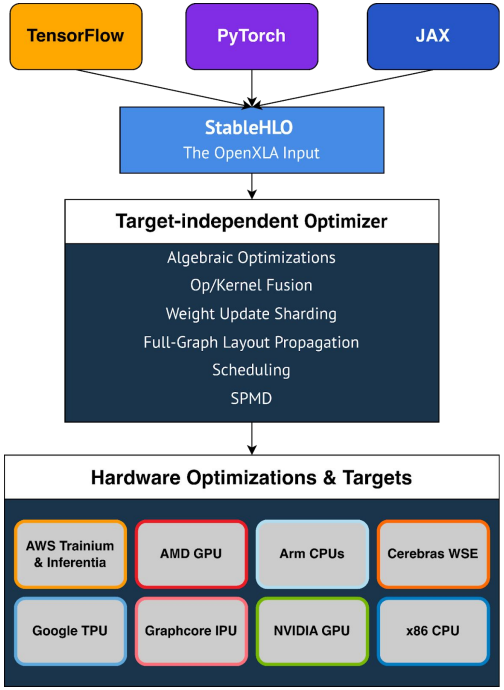
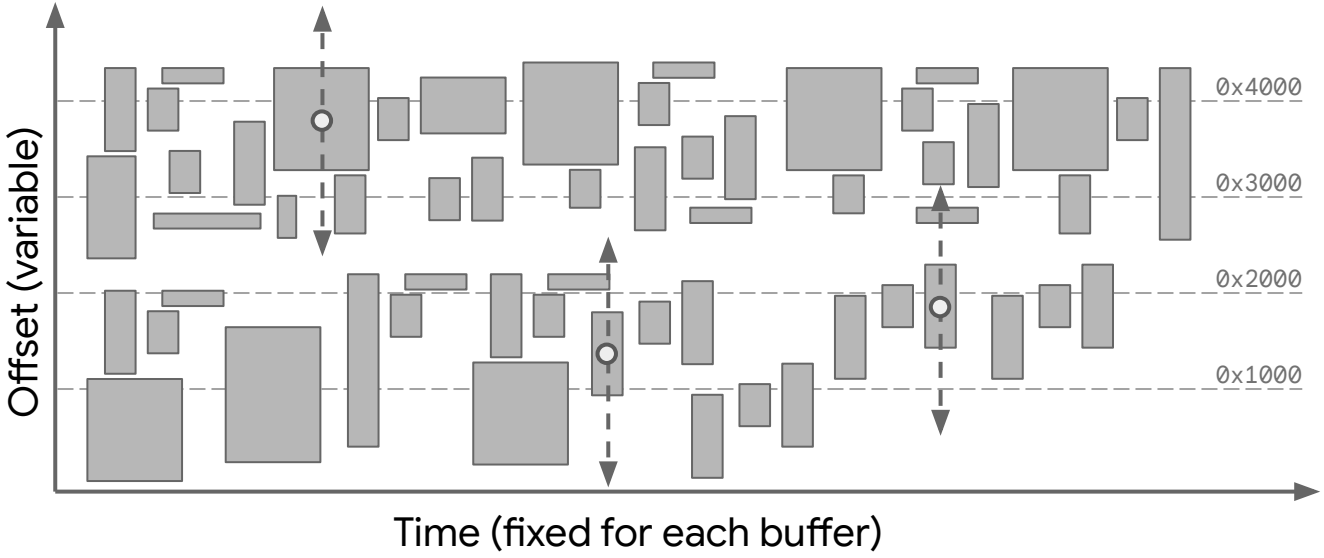


Image Credit:
https://opensource.googleblog.com/2023/03/openxla-is-ready-to-accelerate-and-simplify-ml-development.html

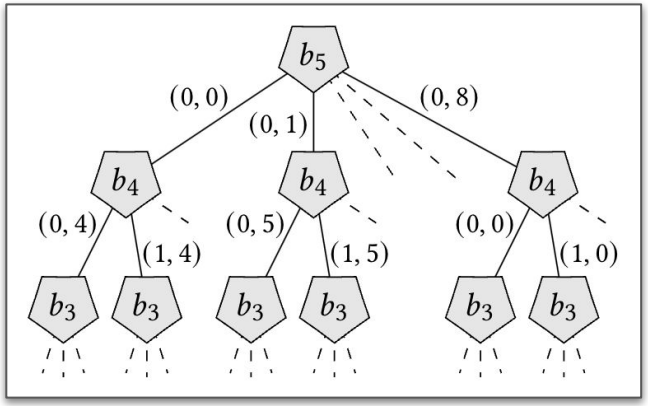
Static Memory Allocation

(Maas, Beaugnon, Chauhan, & Ilbeyi, ASPLOS 2023)



Rectangle Packing as a CSP

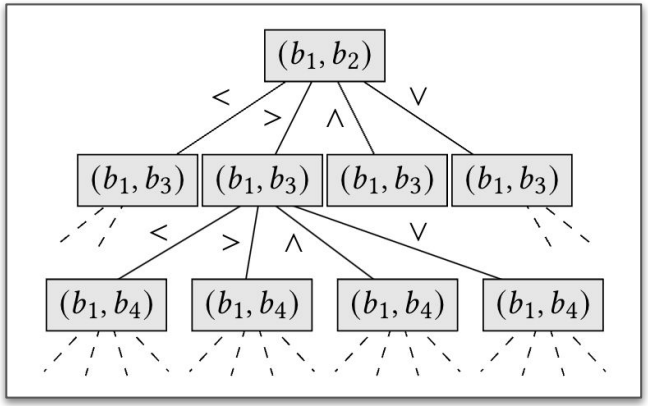
(Korf, ICAPS 2003) + (Korf, ICAPS 2004)



- CSP: Constraint Satisfaction Problem
- <u>*Variables*</u> are rectangle locations, i.e. (x_i, y_i) for $i \in [1, n]$
- <u>*Values*</u> are Euclidean coordinates, e.g. (0, 0) or (0, 1)
- Constraints are (pairwise) non-overlapping conditions

Rectangle Packing as a Meta-CSP

(Moffitt & Pollack, ICAPS 2006)

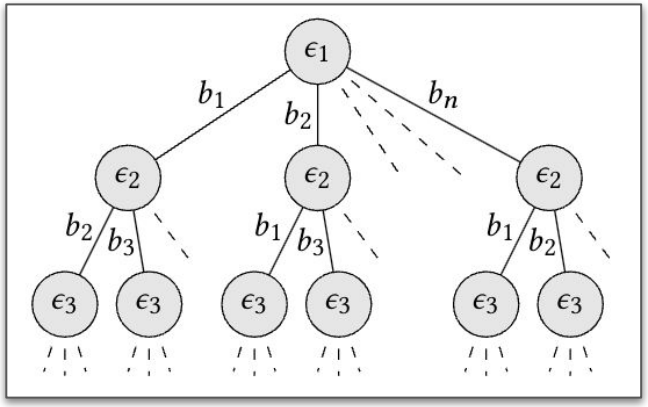


- <u>*Variables*</u> represent rectangle pairs, e.g. rectangles 1 & 3
- <u>*Values*</u> are pairwise spatial relationships, e.g., *left* or *above*
- Consistency checking can be achieved by casting all inequalities into the edges of a *directed acyclic graph*

- Nice property: worst-case complexity of $O(4^p n^2)$ is **completely independent of grid size!**

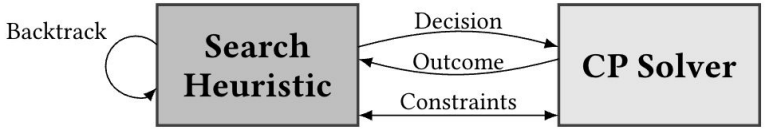
TelaMalloc: Efficient On-Chip Memory Allocation

(Maas, Beaugnon, Chauhan, & Ilbeyi, ASPLOS 2023)



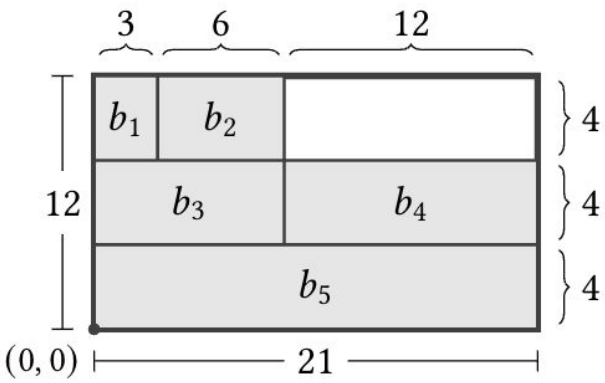
- Exhaustively explores the space of *permutations*, branching on the selection of buffers in a total ordering

- Decouples the (domain-specific) search heuristics from the SAT-based constraint resolution system:



- Empirically outperforms the previous state-of-the-art (solves more problems, orders of magnitude faster, etc.)

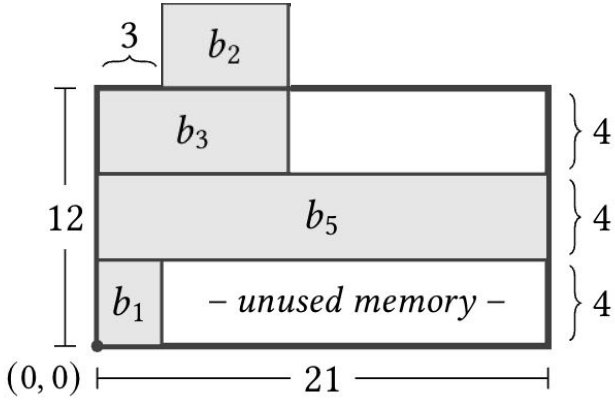
Challenges in the Tela-CSP



$$\mathcal{S}_1 = b_5(0) \rightarrow b_3(4) \rightarrow b_1(8) \rightarrow b_2(8) \rightarrow b_4(4)$$
$$\mathcal{S}_2 = b_5(0) \rightarrow b_3(4) \rightarrow b_1(8) \rightarrow b_4(4) \rightarrow b_2(8)$$
$$\mathcal{S}_3 = b_5(0) \rightarrow b_3(4) \rightarrow b_2(8) \rightarrow b_1(8) \rightarrow b_4(4)$$
$$\mathcal{S}_4 = b_5(0) \rightarrow b_3(4) \rightarrow b_2(8) \rightarrow b_4(4) \rightarrow b_1(8)$$
$$\mathcal{S}_5 = b_5(0) \rightarrow b_3(4) \rightarrow b_4(4) \rightarrow b_1(8) \rightarrow b_2(8)$$
$$\mathcal{S}_6 = b_5(0) \rightarrow b_3(4) \rightarrow b_4(4) \rightarrow b_2(8) \rightarrow b_1(8)$$
$$\mathcal{S}_7 = b_5(0) \rightarrow b_4(4) \rightarrow b_3(4) \rightarrow b_1(8) \rightarrow b_2(8)$$
$$\mathcal{S}_8 = b_5(0) \rightarrow b_4(4) \rightarrow b_3(4) \rightarrow b_2(8) \rightarrow b_1(8)$$

Redundancy in the solution space carries a potential risk of superfluous search

Challenges in the Tela-CSP (cont'd)



- Intermediate solutions can be locally consistent (i.e., within the memory capacity) but ultimately unresolvable

- Ideally these partial assignments would be pruned, resulting in immediate backtracking

- Unfortunately, the SAT-based encoding makes this very difficult to achieve with mere binary non-overlap constraints

Our Approach

The Search Lattice

- Identifies isomorphisms via *offset monotonicity* and *index monotonicity*
- Search restricted to a subset of *canonical solutions* only

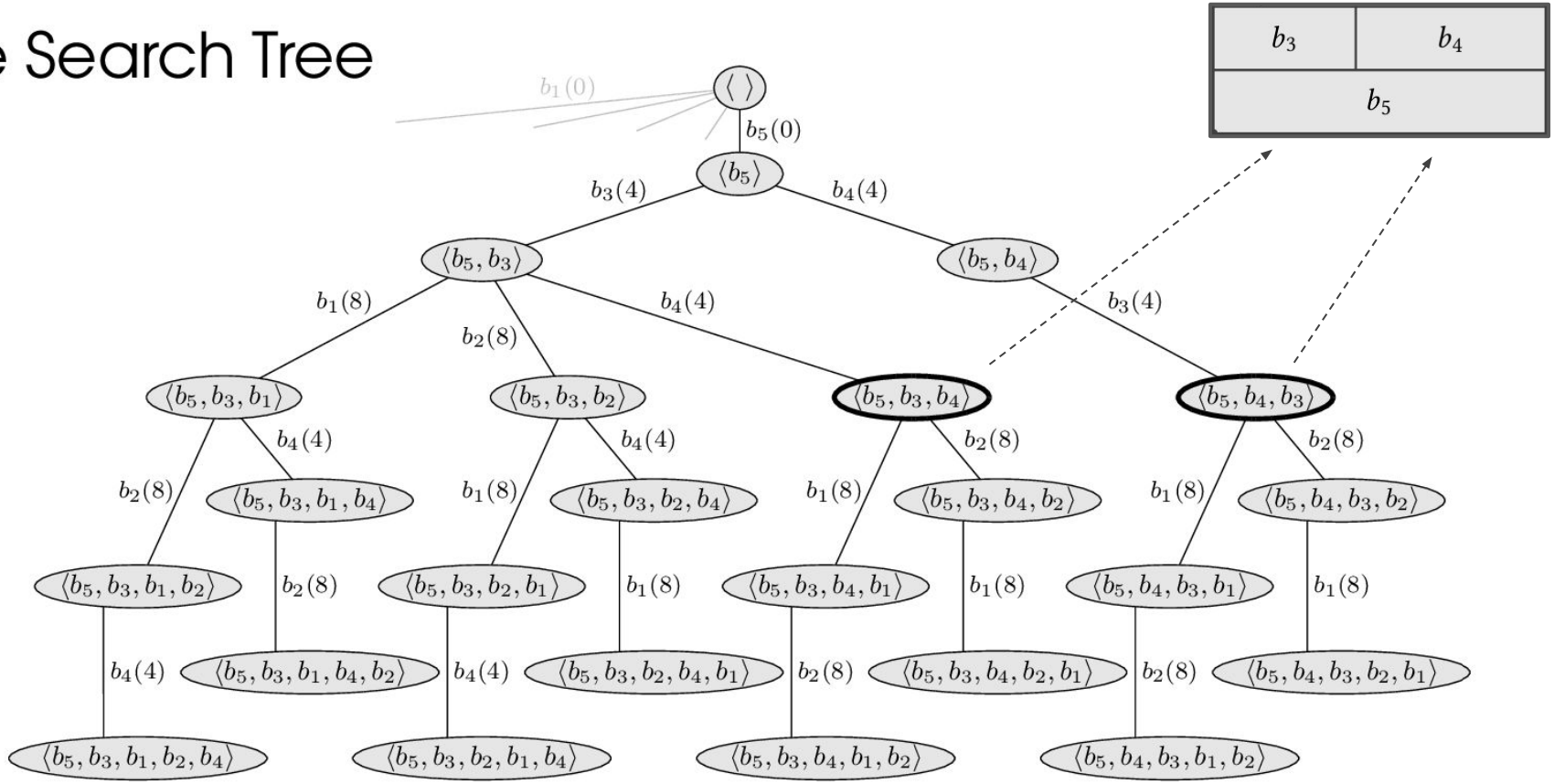
Cross-Section Inference

- Strengthens lower bounds on solution "slack"
- Helps to afford the early detection of inevitable spatial inconsistencies

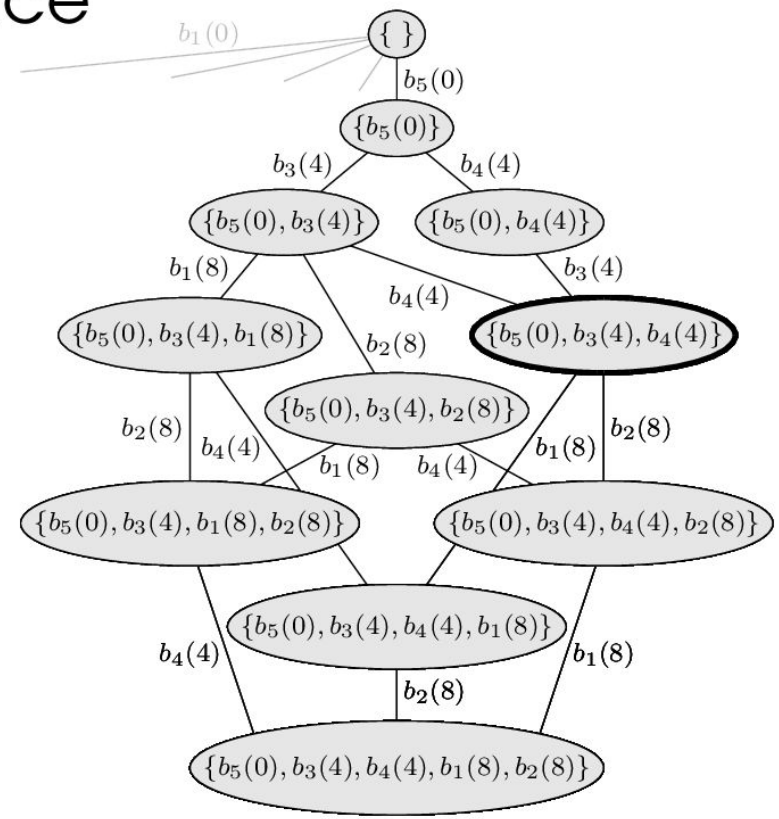
Dominance Detection

- Establishes an upper bound on buffer placement values
- Avoids the generation of solutions that are provably inferior

The Search Tree



The Search Lattice



Attributes of an Allocation

Definition: Two solutions $b_{\epsilon_1}(o_{\epsilon_1}) \rightarrow \cdots \rightarrow b_{\epsilon_n}(o_{\epsilon_n})$ and $b_{\epsilon'_1}(o_{\epsilon'_1}) \rightarrow \cdots \rightarrow b_{\epsilon'_n}(o_{\epsilon'_n})$ are said to be *isomorphic* if and only if:

$$o_i = o'_i \qquad \forall i \in [1, n]$$

Otherwise, they are said to be *non-isomorphic*.

Definition: A solution $b_{\epsilon_1}(o_{\epsilon_1}) \rightarrow \cdots \rightarrow b_{\epsilon_n}(o_{\epsilon_n})$ is said to be *offset monotonic* if and only if:

$$o_{\epsilon_i} \leq o_{\epsilon_j} \qquad \forall i, j : i < j$$

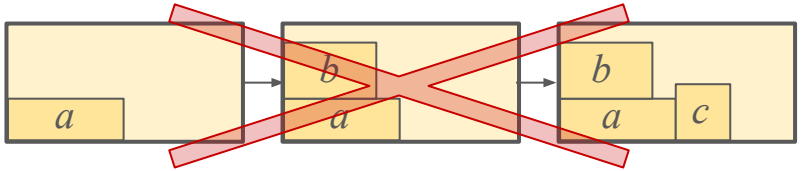
Otherwise, it is said to be *offset non-monotonic*.

Definition: A solution $b_{\epsilon_1}(o_{\epsilon_1}) \rightarrow \cdots \rightarrow b_{\epsilon_n}(o_{\epsilon_n})$ is said to be *index monotonic* if and only if:

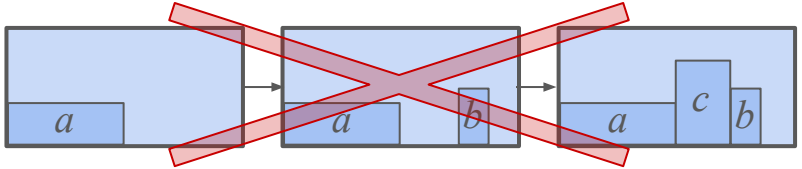
$$o_{\epsilon_i} = o_{\epsilon_j} \Rightarrow \epsilon_i < \epsilon_j \qquad \forall i, j : i < j$$

Otherwise, it is said to be *index non-monotonic*.

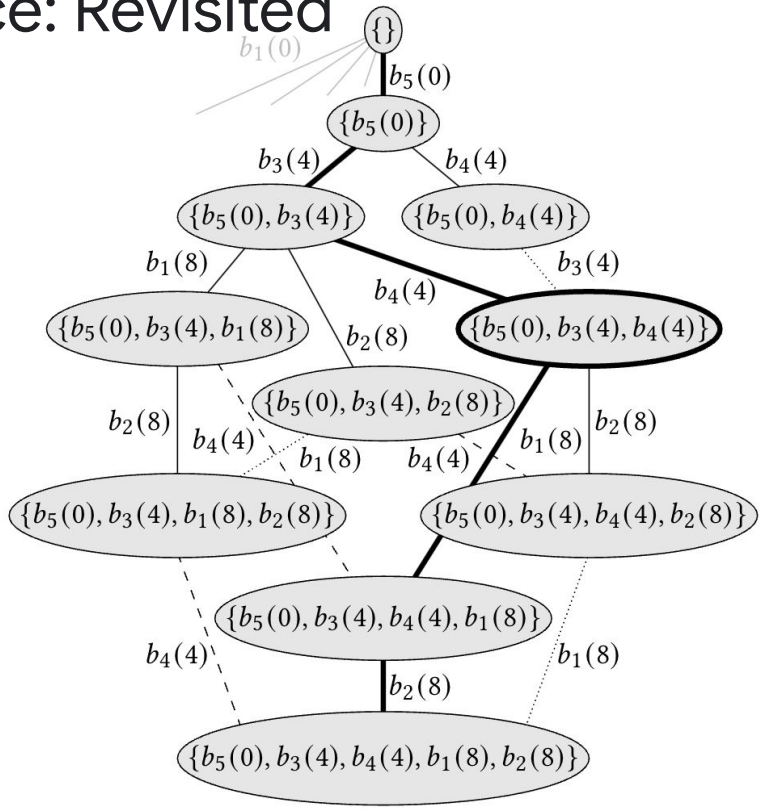
Rule #1: Lower blocks must be placed first



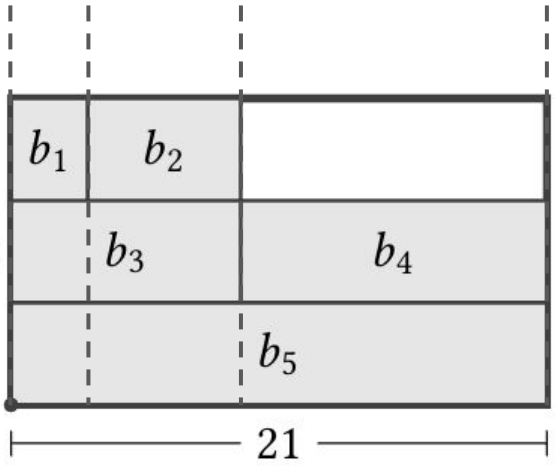
Rule #2: Place blocks at the same offset left-to-right



The Search Lattice: Revisited



Cross Section Inference



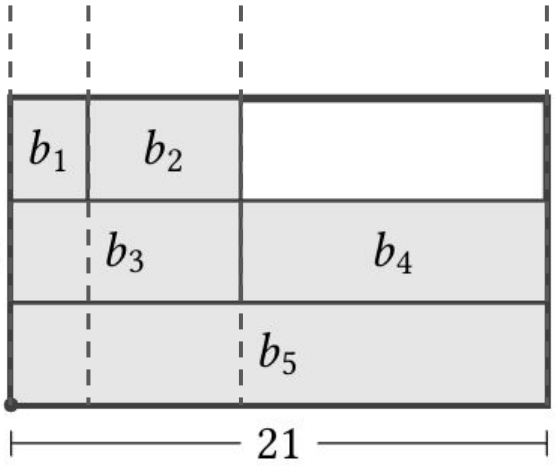
- Define a *section* to be a temporal interval $[s, e)$ such that the set of active buffers is identical at any $t \in [s, e)$

- Section boundaries can be determined prior to search

- Two separate ways of establishing feasibility:

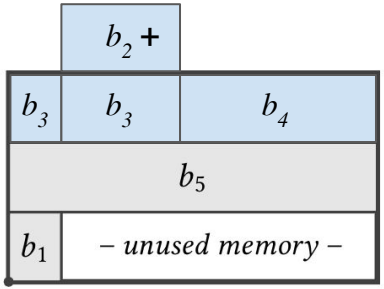
$$\max_{[s,e)} \Big[\underbrace{\sum_j z_j : b_j \in \mathcal{X}_{[s,e)}}_{\text{All buffers in } \mathcal{X}_{[s,e)}} \Big] \leq C \qquad \max_{[s,e)} \Big[\underbrace{\max_i [o_i + z_i] : b_i(o_i) \in \mathcal{S}, b_i \in \mathcal{X}_{[s,e)}}_{\text{Height of cross section } \mathcal{X}_{[s,e)} \text{ in } \mathcal{S}} \Big] \leq C$$

Cross Section Inference (cont'd)

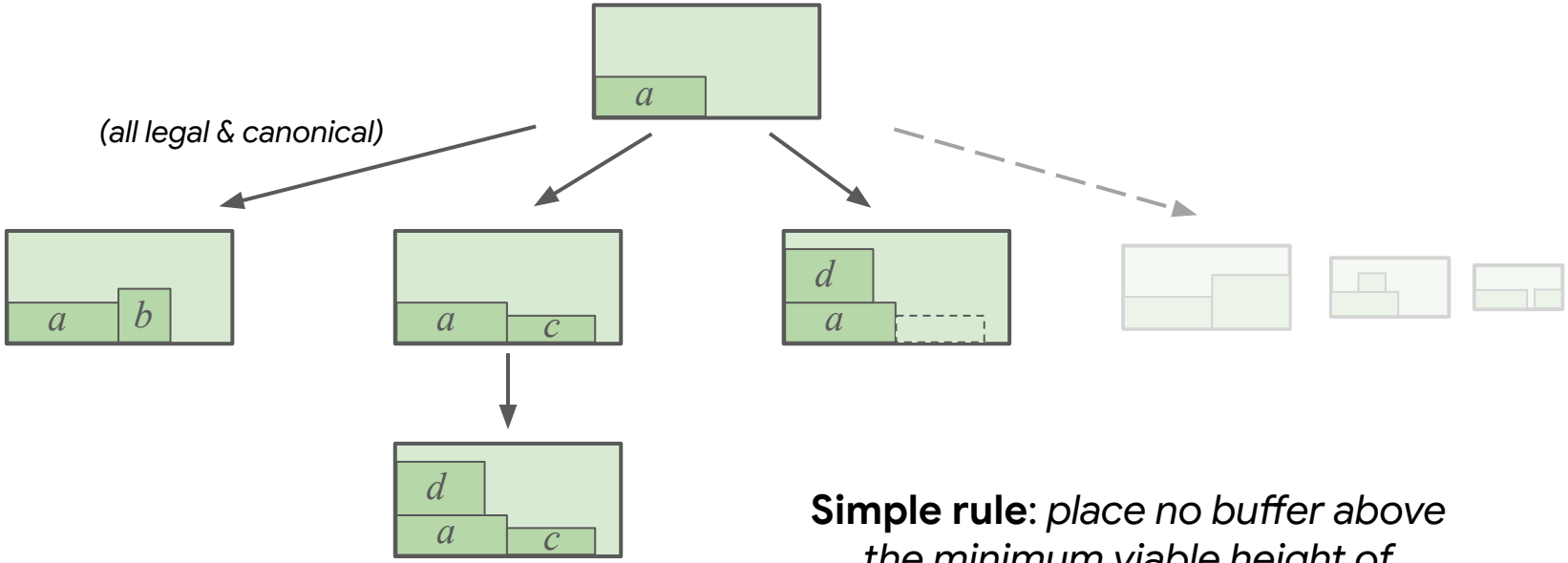


- MiniMalloc combines these two checks into one:

$$\max_{[s,e)} \left[\underbrace{\max_i [o_i + z_i] : b_i(o_i) \in \mathcal{S}, b_i \in \mathcal{X}_{[s,e)}}_{\text{Height of cross section } \mathcal{X}_{[s,e)} \text{ in } \mathcal{S}} + \underbrace{\sum_j z_j : b_j \notin \mathcal{S}, b_j \in \mathcal{X}_{[s,e)}}_{\text{Unallocated buffers in } \mathcal{X}_{[s,e)}} \right] \leq C$$

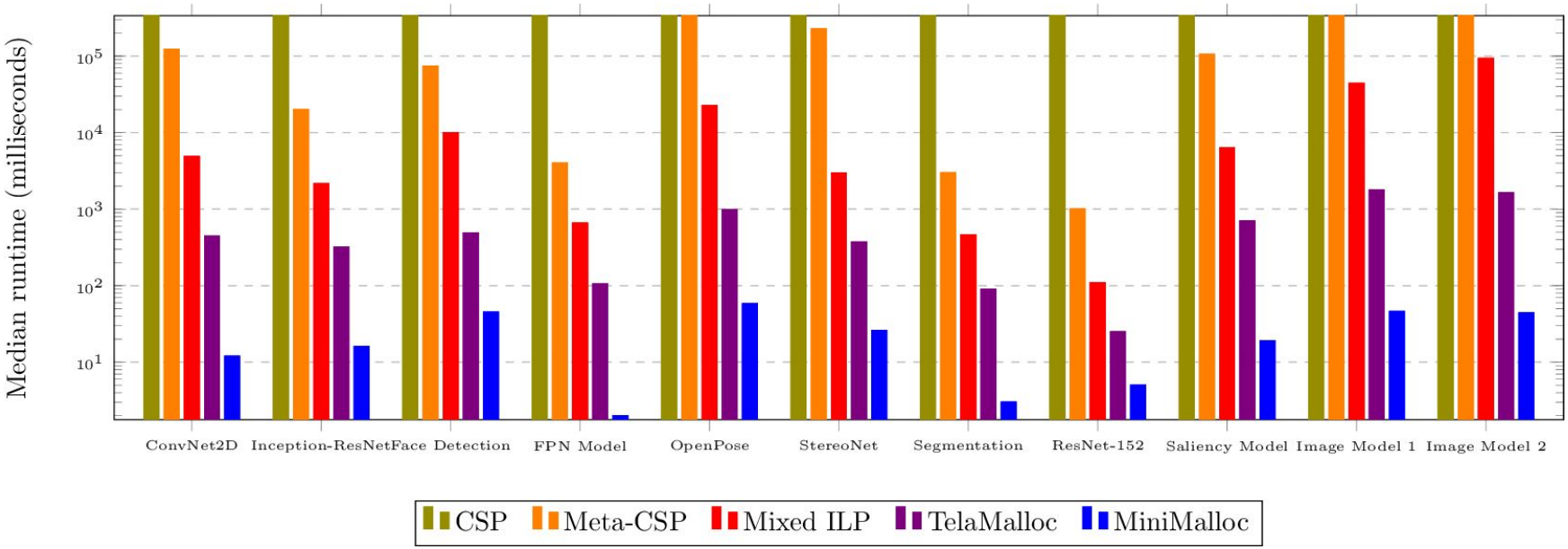


Dominance Detection

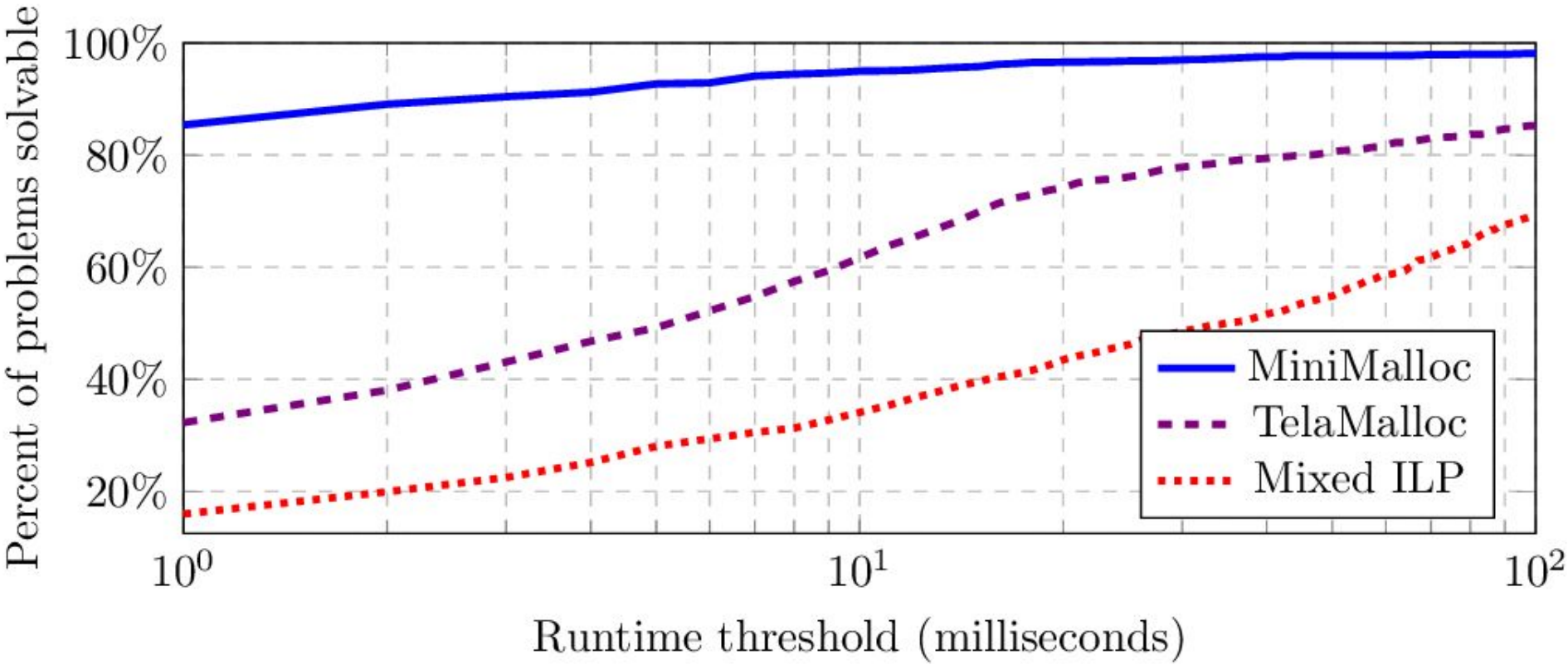


Simple rule: *place no buffer above the minimum viable height of any unallocated buffer*

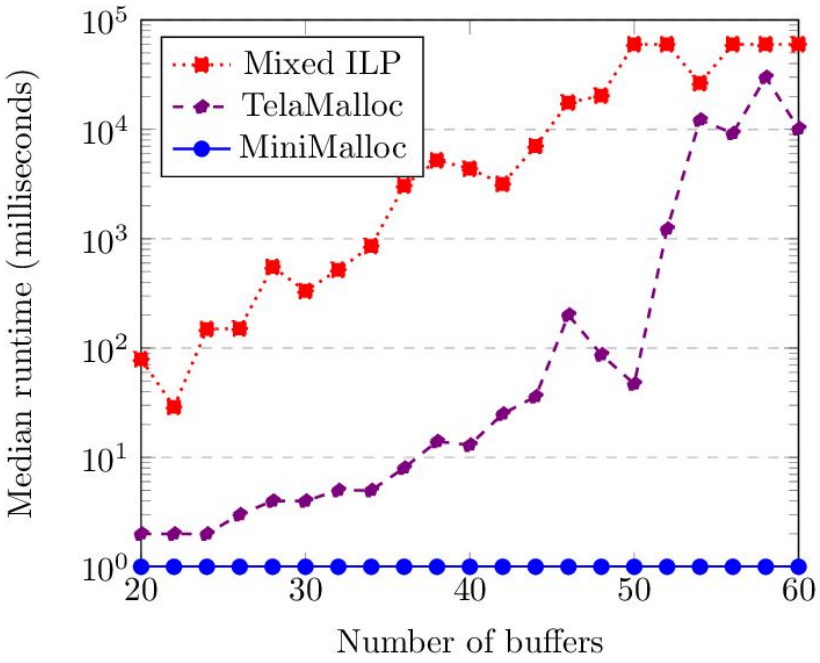
Empirical Results: Original Suite

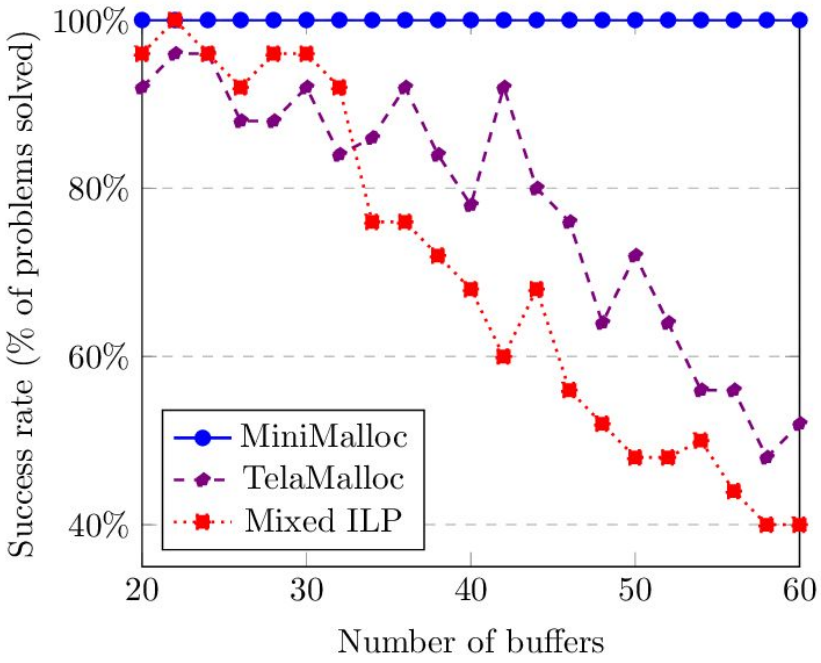


Empirical Results: Extended Suite

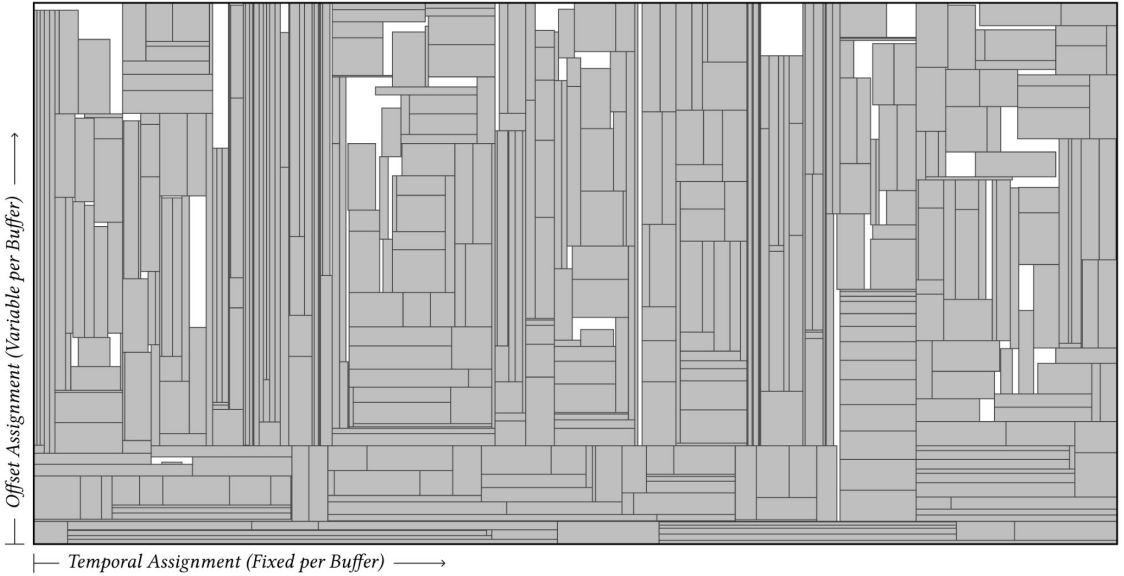


Empirical Results: Synthetic Suite





Empirical Results: Challenging Suite



Benchmark	Mixed ILP	TelaMalloc	**MiniMalloc**
challenging-A	> 3600000 ms	> 3600000 ms	**4910 ms**
challenging-B	> 3600000 ms	> 3600000 ms	**3661 ms**
challenging-C	> 3600000 ms	> 3600000 ms	**637 ms**
challenging-D	> 3600000 ms	> 3600000 ms	**2674 ms**
challenging-E	> 3600000 ms	> 3600000 ms	**3717 ms**
challenging-F	> 3600000 ms	> 3600000 ms	**5358 ms**
challenging-G	> 3600000 ms	> 3600000 ms	**1181 ms**
challenging-H	> 3600000 ms	> 3600000 ms	**1808 ms**
challenging-I	> 3600000 ms	> 3600000 ms	**3908 ms**
challenging-J	> 3600000 ms	> 3600000 ms	**883 ms**
challenging-K	> 3600000 ms	> 3600000 ms	**628 ms**

MiniMalloc is Open Source!

minimalloc Public

Edit Pins | Unwatch 5 | Fork 7 | Starred 86

main | 1 Branch | 1 Tags | Go to file | Add file | Code

mmoffitt Merge pull request #7 from Zhao-Dongyu/main — 987b3c1 · 2 months ago — 111 Commits

benchmarks	challenge -> challenging	7 months ago
external	Initial commit.	8 months ago
img	Replaces the order of ACM badges in the logo	2 months ago
scripts	Updates the script to use the proper benchmark directory.	7 months ago
src	Add display of buffer.id in print solutions in LaTeX form	2 months ago
tests	Adds an additional field in Buffer for 'hint'	5 months ago
.gitmodules	Changes googletest submodule to use HTTPS instead of ...	3 months ago
CMakeLists.txt	Moves the Overlap() logic into a new effective_size() met...	7 months ago
CONTRIBUTING.md	Initial commit.	8 months ago
LICENSE	Initial commit.	8 months ago
README.md	Adds page numbers to BibTex citation	2 months ago

README | Code of conduct | Apache-2.0 license | Security

mini malloc

A Lightweight Memory Allocator for Hardware-Accelerated Machine Learning

About

A lightweight memory allocator for hardware-accelerated machine learning

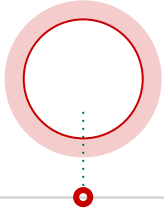
machine-learning | compiler | static-analysis | memory-allocation | hardware-acceleration | combinatorial-optimization | rectangle-packing | asplos | asplos23

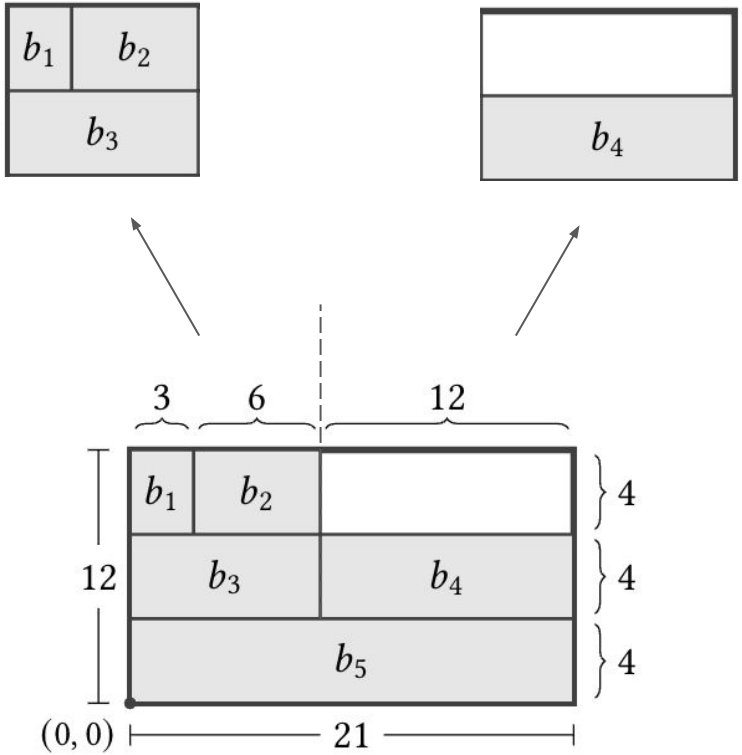
Readme
Apache-2.0 license
Code of conduct
Security policy
Activity
Custom properties
86 stars
5 watching
7 forks
Report repository

Releases 1

v0.1.0 Latest
on Oct 31, 2023

The Fourth Technique: Dynamic Decomposition



Dynamic Decomposition

- Continually check for "cutlines" that separate groups of buffers
- Solve such problems independently